

Python Flow Control

- ❖ if....else
- ❖ for loop
- ❖ while loop
- ❖ break and continue

if.....else

- The if else statement is used in Python for decision making.

```
if test expression:
    body of if
else:
    body of else
```

- To test more than one condition following syntax is used-

```
if test expression:
    body of if
elif test expression:
    body of elif
else:
    body of else
```

```
print("Enter the Value of X")
x=int(input(">"))

if x >= 5:
    print("The value is greater than 5")
else:
    print("The value is Less than 5")
```

```
Enter the Value of X
>6
The value is greater than 5
```

```
print("Enter the Value of X")
x=int(input(">"))

if x > 5:
    print("The value is greater than 5")
elif x<5:
    print("The value is Less than 5")
else:
    print("The value is equal to 5")
```

```
Enter the Value of X
>3
The value is Less than 5
```

Dangling else Problem

- The dangling else problem happens when there are nested if statements without braces, and it becomes unclear to which if statement the else belongs.

```
#include<stdio.h>
int main()
{
    int num;
    printf("enter the number");
    scanf("%d",&num);
    if (num<5)
        if (num<2)
            printf("Number is less than 2");
    else
        printf("Number is grater than 5");
}
```

If num=3
O/P:

```
#include<stdio.h>
int main()
{
    int num;
    printf("enter the number");
    scanf("%d",&num);
    if (num<5)
        if (num<2)
            printf("Number is less than 2");
        else
            printf("Number is grater than 5");
}
```

If num=3
O/P:

Else always matches the closest if

Dangling else Problem

- The dangling else problem happens when there are nested if statements without braces, and it becomes unclear to which if statement the else belongs.

```
num= int(input("Enter the number"))
if (num<5):
    if (num<2):
        print("Number is less than 2")
    else:
        print("Number is greater than 5")
```

If num=3
O/P:

If num=3
O/P: Number is greater than 5

```
num= int(input("Enter the number"))
if (num<5):
    if (num<2):
        print("Number is less than 2")
else:
    print("Number is greater than 5")
```

If num=3
O/P:

If num=3
O/P: No output

Ternary Operator

In C

Condition ? Expression if True : Expression if False

```
int max = a > b ? a : b
```

In Python:

Expression if True if Condition else Expression if False

```
max = a if a > b else b
```

While loop

- ❖ A while loop is used to execute a block of statements repeatedly until a given condition is satisfied.
- ❖ When the condition becomes false, the line immediately after the loop in the program is executed.
- ❖ We generally use this loop when we don't know beforehand the number of times to iterate.

Syntax :

while expression:

Body of while

```
: count = 0
  while (count < 3):
    count = count + 1
    print("Hello Geek")
```

```
Hello Geek
Hello Geek
Hello Geek
```

While-else statement

```
a= int(input("Enter the value of a"))
while a>2:
    print(a)
    a=a-1
else:
    print("Condition unsatisfied")
```

If a=1

O/P:

```
Enter the value of a2
Condition unsatisfied
```

If a=5

O/P:

```
Enter the value of a5
5
4
3
Condition unsatisfied
```

```
a= int(input("Enter the value of a"))
while a>2:
    print(a)
    a=a-1
    if a==4:
        break
else:
    print("Condition unsatisfied")
```

If a=8

O/P:

```
Enter the value of a8
8
7
6
5
```

For loop

- ❖ A for loop is used for iterating over a sequence (that is, either a list, a tuple, a dictionary, a set, or a string).

Syntax :

for val in sequence:

Body of for

- ❖ Here, val is the variable that takes the value of the item inside the sequence on each iteration.

- ❖ Loop continues until we reach the last item in the sequence.

Program to find the sum of all numbers stored in a list.

```
numbers=[1,2,3,4,5,6,7,8,9]
totalsum=0
for val in numbers:
    totalsum=totalsum+val
print("The sum is: ", totalsum)
```

The sum is: 45

For loop

Iterating over a string

```
name= "Radheshyam"  
for i in name:  
    print(i)
```

Iterating over a Sequence

```
name= ["Ram", "Shyam", "Mohan"]  
for i in name:  
    print(i)
```

Iterating by sequence index

```
name= ["Ram", "Shyam", "Mohan"]  
for i in range(0,3):  
    print(name[i])
```

Iterating with item and index

```
name= ["Ram", "Shyam", "Mohan"]  
for i,j in enumerate(name):  
    print(i,j)
```

0	Ram
1	Shyam
2	Mohan

range() function

It is used to generate a sequence of numbers

Syntax: range(start, stop, step)

```
for i in range(5):  
    print(i)
```

```
for i in range(0,5):  
    print(i)
```

```
for i in range(1,5):  
    print(i)
```

```
for i in range(0,10,2):  
    print(i)
```

```
for i in range(10,0,-1):  
    print(i)
```

for-else statement

The else block runs only if the loop finishes normally.

Syntax: `range(start, stop, step)`

```
for i in range (5):  
    print(i)  
else:  
    print("Loop Completed")
```

```
for i in range (6):  
    if i==3:  
        break  
    print(i)  
else:  
    print("Loop Completed")
```

Break, continue, and pass statements

- ❖ Loop iterates over a block of code until the text expression is false, but sometimes we wish to terminate the current iteration or even the whole loop without checking text expression.

Break:

- The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.
- If break statement is inside a nested loop, break will terminate the inner most loop.

```
for i in range(0,5):  
    print("enter the value of x")  
    x=int(input(">"))  
    if (x>9):  
        break  
print("The End")
```

```
enter the value of x  
>10  
The End
```

Continue:

- The continue statement is used to skip the rest of the code inside the loop for the current iteration only.
- Loop does not terminate but continues with the next iteration.

```
: for i in range(0,5):  
    print("enter the value of x")  
    x=int(input(">"))  
    if (x>9):  
        continue  
    print("The value is less then 9")  
print("The End")
```

```
enter the value of x  
>3  
The value is less then 9  
enter the value of x  
>10  
enter the value of x  
>|
```

Pass:

Python does not allow an empty block.

Exercise- 6

Print the following pattern

```
*
* *
* * *
* * * *
* * * * *
```

```
: for i in range(0,5):
    for j in range(0,i+1):
        print("*",end=" ")
    print("\n")
```

```
*
* *
* * *
* * * *
* * * * *
```

Print the following pattern

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

```
for i in range(1,6):
    for j in range(1,i+1):
        print (j, end=" ")
    print("\n")
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Exercise- 4

Write a program to give a discount of 10% if the total bill amount exceeds 1000 Rs.

```
: shopping_amount=int(input("Enter the shoping amount: "))  
  
if(shopping_amount>1000):  
    discount=(10/100)*shopping_amount  
    print("Discount =", discount)  
    shopping_amount=shopping_amount-discount  
print("final shopping amount=", shopping_amount)
```

```
Enter the shoping amount: 2000  
Discount = 200.0  
final shopping amount= 1800.0
```

Exercise- 5

Write a program to print the number of digits of a given number using while loop.

```
: num=int(input("Enter a number: "))
  digits=0
  while num!=0:
      num = num//10
      digits=digits+1
  print("Number of digits =", digits)
```

Enter a number: 123456

Number of digits = 6

Exercise- 7

Write a program that loops through numbers from 1 to 20. Print each number, but:

- If the number is 10, skip it using continue.
- If the number is 15, exit the loop using break.

```
for num in range(1, 21):
    if num == 10:
        print("Skipping 10...")
        continue
    elif num == 15:
        print("Found 15! Exiting loop...")
        break
    print(num)
```

```
1
2
3
4
5
6
7
8
9
Skipping 10...
11
12
13
14
Found 15! Exiting loop...
```